Subtyping and Wildcards

13 July 2025 11:16

Subtyping and the Substitution Principle

Subtyping is a key feature of object oriented languages such as Java. In Java, one type is a subtype of another if they are related by an extent or implements clause. Here are some examples :

Integer	Is a subtype of	Number
Double	Is a subtype of	Number
ArrayList<E>	Is a subtype of	List<E>
List<E>	Is a subtype of	Collection<E>
Collection<E>	Is a subtype of	Iterable<E>

Subtyping is transitive meaning. That if one type is a subtype of a 2nd and 2nd is the subtype of 3rd, then the 1st is a subtype of the 3rd. So from the last two lines in the preceding list, it follows that List<E> is a subtype of Iterable<E>. If one type is a subtype of another, we also say that the 2nd is the supertype of the 1st. Every reference type is a subtype of Object and Object is a super type of every reference type. We also define the subtype relationship so that every type is a subtype of itself.

The most important property of subtyping is summarized in a substitution principle.
Substitution Principle: Wherever a value of type T is expected, you can provide instead of value of a subtype of T.
For example, a variable of a given type may be assigned a value of any subtype of that type, and a method with a parameter of A given type may be invoked with an argument of any subtype of that type.

Consider the interface Collection<E>. One of its method is add which takes the parameter of type E.

```
interface Collection<E> extends iterable<E>{
    public boolean add(E element);
    …
}
```

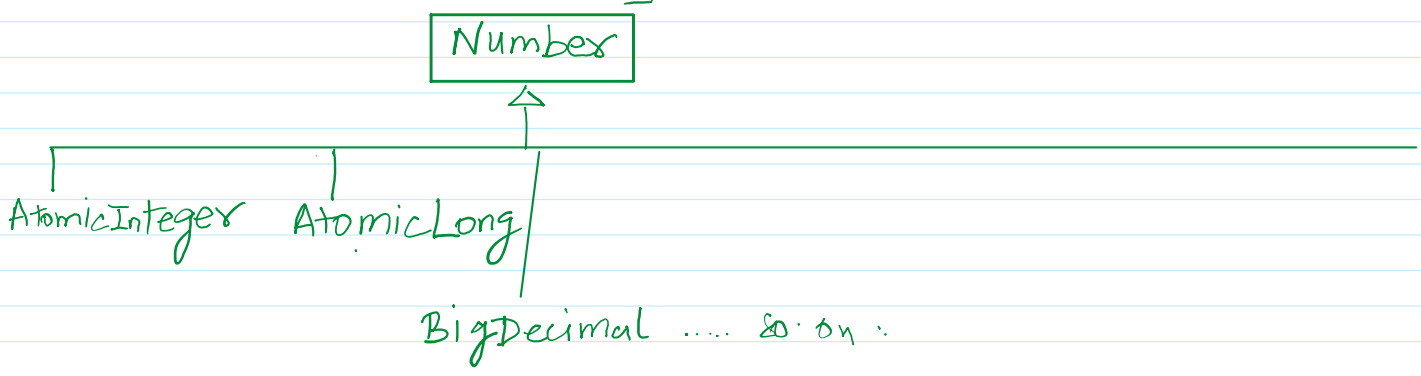
According to the substitution principle, if we have a collection of numbers, we may add an Integer or a Double to it, because Integer and Double are the subtypes of Number.

```
List<Number> nums = new ArrayList<>();
nums.add(2);
nums.add(0.25);
Assert num.equals(List.of(2, 0.25)):
```

public abstract class Number extends Object implements Serializable

Subclasses of Number.

[AtomicInteger, AtomicLong, BigDecimal, BigInteger, Byte, Double, DoubleAccumulator, DoubleAdder, Float, Float16, Integer, Long, LongAccumulator, LongAdder, Short].



```
List<Number> nums = new ArrayList<>();
 nums.add(2);
 nums.add(0.25);
Assert num.equals(List.of(2, 0.25)):
```

Here subtyping is used in two ways for each method call. The first call permits because numbs has type List<Number>, Which is a subtype of Collection <Number>. And 2 has type Integer(thanks to boxing), Which is a subtype of Number. The second call is similar permitted in both the calls the E in List<E> is taken to be number. It maybe seem reasonable to expect that since Integer is a subtype of Number, it follows that List<Integer> is a subtype of List<Number>. But this is not the case, because if it were, the substitution principle would rapidly get us into trouble. To see why it is not safe to assign a value of type List<Integer> to a variable of type List<Number>. Consider the following code:

```
List<integer> ints = new ArrayList<>();
List<Number> nums = ints;  //1. can't be allowed
nums.add(3.14);
```

This code assigns the variable ints to point to a list of Integer and then. If the substitution principle were required to succeed. Assigns nums to point to the same list. In that case, call in the following line, which is clearly legal, would add a Double to this list, meaning that the variable ints typed as List<Integer> would be pointing at List containing a double and Java type System would be broken. Obviously this can't be allowed. The solution is to outlaw assignment 1.
If List<Integer> is prevented from being a subtype of List<Number>, then the substitution principle does not apply and one can be reported as a compiler error.

What about reverse? Can we take List<Number> to be a subtype of List<Integer>? No that doesn't work either. Its shown by the following code.

```
List<Number> ints = new ArrayList<>();
nums.add(3.14);
List<Integer> nums = ints;  //2. can't be allowed
```

We're allowing 2 leads to the same problem once again. A variable type as List<Integer> is pointing to a list containing a Double. So List<Number> is not a subtype of List<integer>. And since we have already seen that List<Integer>. Is not a subtype of List<Number>. And all we have is the trivial case where List<Integer> is a subtype of itself. (We also have more intuitive relationship that List<Integer> is a subtype of Collection<Integer>.)

Arrays behave quite differently with them. Integer. [] Is a subtype of Number [] We will compare the treatment of lists and arrays, Later.

Sometimes we would like List to behave more like Aries in that we want to accept not only a list with element of a given type, but also a list with elements of any subtype of a given type. For this purpose we use wild cards.

Wildcards with extends
Another method in Collection interface is addAll(), which adds all the members of one collection to another.

```
interface Collection<E> implements iterable<E>{
   …
    public boolean addAll(Collection<? extends E> C);
   …
}
```

superclass class

Clearly, given a collection of elements of type E, we should be able to Add all the members of another collection with elements of type E. The quizzical phrase "? extends E" Means that it is also OK to Add all members of a collection with elements of any type that is a subtype of E. The question mark is called a wild card. It stands for some as-yet-unknown subtype of E.
Here is an example. We create an empty list of numbers and add to it, first a list of integers and then a list of doubles.

```
List<Number> nums = new ArrayList<>();
List<Integer> ints = List.of(1, 2);
List<Double> dbls = List.of(1.0, 0.5);
nums.addAll(ints);
nums.addAll(dbls);
```

```
assert nums.equals(List.of(1, 2, 1.0, 0.5));
```

The first call is permitted because nums has type List<Number>. Which is a subtype of Collection<Number>. And it has a type List<Integer> which could which is a subtype of Collection<Integer>. Which is in turn a subtype of Collection<? extends Number>.
The second call is similarly permitted. In both the calls E is taken to be no. If the parameter declaration for Add all had been written without the wild card, then the calls to add list of integers and double S to a list of numbers would have not been permitted. You would only have been able to add list that was explicitly declared to be a list of numbers.
We can also use wild cards when declaring variables. Here is a variant of the introductory example of the preceding section changed by adding a wild card to the 2nd line:

```
List<Integer> ints = new ArrayList<>();
List<? extends Number> nums = ints; //3 now legal
nums.add(3.14); //4 can't be allowed
```

In general, if a structure contains element with a type of form "? extends E", We can get elements out of the structure, but we cannot put elements into the structure. To put elements into the structure, we need another kind of wild card.

Wildcards with super

Here's a method from the convenience class Collections, that copies the element from a source list into a destination list:

```
public static<T> void copy(List<? super T> dst, List<? extends T> src){
    for(int i = 0; i < src.size(); i++){
        dst.set(I, src.get(i));
    }
}
```

The quizzical phrase "? super T" means that the destination list have many elements of any type that is a super type of T, just as the source list may have elements of any type that is a subtype of T.

```
jshell> import java.util.*;

jshell> List<Object> objs = Stream.of(2, 3.14, "four").collect(Collectors.toList());
objs ==> [2, 3.14, four]
```

of type Number

```
jshell> List<Integer> ints = List.of(5, 6);
ints ==> [5, 6]
```

Integer

```
jshell> Collections.copy(objs, ints)

jshell> assert objs.equals(List.of(5, 6, "four"));

jshell> objs
objs ==> [5, 6, four]

jshell> ints
ints ==> [5, 6]

jshell>
```

As with any generic method, the type parameter may be inferred or may be given explicitly. In this case, there are four possible choices, all of which type check, and all of which have the same effect.

1. Collection.copy(objs, ints);
2. Collections.<Object>copy(objs, ints);

3. Collections.<Number>copy(objs, ints);
4. Collections.<Integer>copy(objs, ints);

The first call leaves the type parameter implicit. It is taken to be Integer since that is the most specific choice that works. In the 3rd line it is taken to be Number. The call is permitted because objs has type List<Object>, which is a subtype of List<? super Number>(Since Object is a super type of Number as required by super wild card) and ints has type List<Integer>, which is a subtype of List<? extends Number>(Since Integer is a subtype of Number as required by the extends wild card.)

We could also declare the method with several possible signatures (A method signature is a combination of the method identifier, its type parameters and the types and the order of its parameter.)

```
static <T> void copy(List<T> dest, List<T> src);
static <T> void copy(List<T> dest, List<? extends T> src);
static <T> void copy(List<? super T> dest, List< T> src);
static <T> void copy(List<? super T> dest, List<? extends T> src);
```

The first of these is too restrictive as it only permits call when the destination and source have exactly the same type.
The remaining three are equivalent for calls that use implicit type parameters, but differ for explicit type parameters. For the example calls above, the second signature works only when the type parameter is Object. The third signature works only when the type parameter is integer. And the last signature works. For all three type parameters, that is Object, Number and Integer. When writing a method signature, always use wild cards where you can since. This permits the widest range of calls.

The Get and Put Principle
It may be good practice to insert wild cards whenever possible. But how do you decide which wild card to use? Where should you use extents, where should you use super, and where it is inappropriate to use a wild card at all?

Fortunately, a simple principle determines which is appropriate.
The get and put principal: Use an **extends** wild card when you only get values out of structure, Use a **super** wild card when you only put values into a structure and don't use a wild card when you both get and put.

Joshua Bloch gives this principle the mnemonics PECS(producer extends, consumer super). We already saw this principle at work in the signature of the copy method:

```
public static <T> void copy(List<? super T> dst,  List<? extends T> src);
```

The method **gets** values out of the source "src", so it is declared with an **extends** wild card. And it **puts** the values into the destination "dst". So it is declared with a **super** wild card.
Whenever you use an iterator or a stream you are getting values out of a structure, so you must use an **extends** wild card. Here is the iterator version of a method that takes a collection of numbers, converts each to a double, and sums them up.

Iterator

```
public static double sum(Collection<? extends Number> nums){
    double s = 0.0;
    for(Number num : nums){
      s += num.doubleValue();
    }
    return s;
  }
```

foreach

Iterator

The stream equivalent doesn't declare a **Number** variable explicitly, but the type constraint on the argument is exactly the same.

```
package GenericsAndCollections;
import java.util.*;


/**
 * Write a description of class Arithmetic here.
```

```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Arithmetic
{
   public static double sum(Collection<? extends Number> nums){
      double s = 0.0;
      for(Number num : nums){
         s += num.doubleValue();
      }
      return s;
   }

   public static double Sum(Collection<?extends Number> nums){
      return nums.stream().mapToDouble(Number::doubleValue).sum();
   }

   public static void main(String args[]){
      List<Integer> ints = List.of(1,2,3);
      assert sum(ints) == 6.0;

      List<Double> doubles = List.of(2.5, 3.5);
      assert sum(doubles) == 6.0;

      List<Number> nums = List.of(1, 2, 2.5, 3.5);
      assert sum(nums) == 9.0;
   }
}
```

The first two calls would not be legal if **extends** was not used. Whenever you use the sum method you are putting values into a structure, so you should use a **super** wild card.
Here is a method that takes a collection of numbers and an integer n, and puts the first n integers, starting from zero, into the collection:

```
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class Arithmetic here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Arithmetic
{
   public static double sum(Collection<? extends Number> nums){
      double s = 0.0;
      for(Number num : nums){
         s += num.doubleValue();
      }
      return s;
   }

   public static double Sum(Collection<?extends Number> nums){
      return nums.stream().mapToDouble(Number::doubleValue).sum();
   }
```

```
    public static void count(Collection<? super Integer> ints, int n){
        for(int i = 0; i < n; i++){
            ints.add(i);
        }
    }

    public static void main(String args[]){
        System.out.print("\f");
        List<Integer> ints = List.of(1,2,3);
        assert sum(ints) == 6.0;

        List<Double> doubles = List.of(2.5, 3.5);
        assert sum(doubles) == 6.0;

        List<Number> nums = List.of(1, 2, 2.5, 3.5);
        assert Sum(nums) == 9.0;

        List<Integer> ints1 = new ArrayList<>();
        count(ints1, 5);
        System.out.print("\nints1 = "+ints1);

        List<Number> nums1 = new ArrayList<>();
        count(nums1, 5);
        nums1.add(5.0);
        System.out.print("\nnums1 = " + nums1);

        List<Object> objs1 = new ArrayList<>();
        count(objs1, 5);
        objs1.add("Five");
        System.out.print("\nobjs1 = " + objs1);


    }
}
```

These two calls would not be legal if super was not used.

Whenever you both put values and get values out of the same structure, you should not use a wildcard.

```
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class Arithmetic here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Arithmetic
{
    public static double sum(Collection<? extends Number> nums){
        double s = 0.0;
        for(Number num : nums){
            s += num.doubleValue();
        }
        return s;
    }

    public static double Sum(Collection<?extends Number> nums){
        return nums.stream().mapToDouble(Number::doubleValue).sum();
    }
```

```
    public static void count(Collection<? super Integer> ints, int n){
        for(int i = 0; i < n; i++){
            ints.add(i);
        }
    }

    public static double sumCount(Collection<Number> nums, int n){
        count(nums, n);
        return sum(nums);
    }

    public static void main(String args[]){
        System.out.print("\f");
        List<Integer> ints = List.of(1,2,3);
        assert sum(ints) == 6.0;

        List<Double> doubles = List.of(2.5, 3.5);
        assert sum(doubles) == 6.0;

        List<Number> nums = List.of(1, 2, 2.5, 3.5);
        assert Sum(nums) == 9.0;

        List<Integer> ints1 = new ArrayList<>();
        count(ints1, 5);
        System.out.print("\nints1 = "+ints1);

        List<Number> nums1 = new ArrayList<>();
        count(nums1, 5);
        nums1.add(5.0);
        System.out.print("\nnums1 = " + nums1);

        List<Object> objs1 = new ArrayList<>();
        count(objs1, 5);
        objs1.add("Five");
        System.out.print("\nobjs1 = " + objs1);

        List<Number> nums2 = new ArrayList<>();
        double sum = sumCount(nums2, 5);
        System.out.print("\nSum = " + sum);

    }
}
```

The collection is passed to both sum and count, so its element type must extend Number (as sum requires)and be a super type of Integer (as count requires). The only two classes that satisfies both of these constraints are Number and Integer. We have picked the 1st of these.

Since there is no wild card, the argument must be a collection of number.
If you don't like having to choose between Number and Integer. It might occur to you that if Java let you write a wild card with both extents and super. You would not need to choose. For instance, we could write the following:

```
 double sumCount(Collection<? extends Number super Integer>
coll, int n){
…//not legal in Java.
}
```

Then we could call some count on either a collection of numbers or a collection of integers. But Java doesn't permit this. The only reason for outlawing? It is simplicity, and conceivably Java might support such notation in the future. But for now, if you need to both get and put, then don't use wildcards.

The **get and put principal** also works the other way around. If an extends wild card is present, pretty much all of you will be able to do this is get but not put values of that type. And if a super wild card is present, pretty much all of you will be able to do. Is put but not get values of that type. For example, consider the following code, which uses a list declared with an extends wildcard:

```
List<Integer> ints2 = new ArrayList<>();
ints2.add(1);
ints2.add(2);
List<? extends Number> nums3 = ints2;
double dbl = sum(nums);
//nums3.add(3.14);//Compile-time error
```

The call to some is fine because it gets values from the list, but the call to add is not because it puts a value into the list. This is just as well, since otherwise we could add a double to a list of integers.
The exception proves the rule, as the saying goes. And indeed each of these rules has an exception. The one value that you can put into a type declared with an extents wild card is null. The only value that belongs to every reference type.

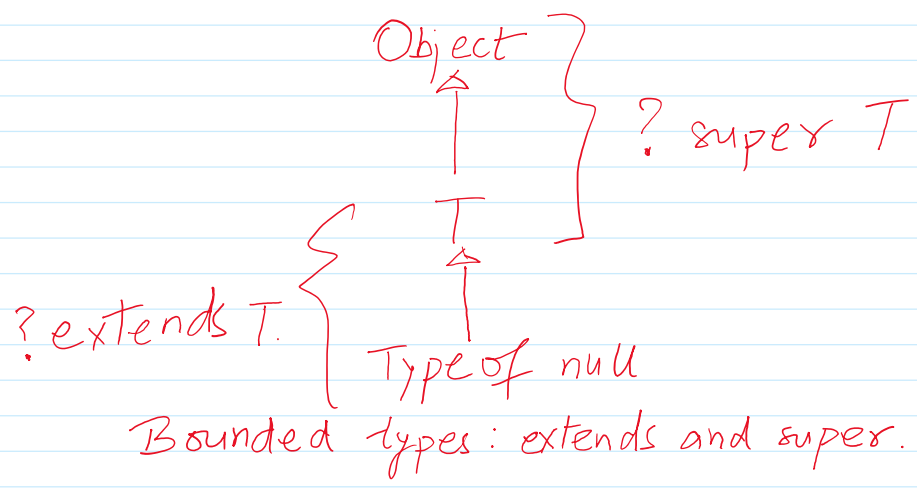
```
List<Integer> ints2 = new ArrayList<>();
    ints2.add(1);
    ints2.add(2);
    List<? extends Number> nums3 = ints2;
    double dbl = sum(nums);

    nums3.add(null); //exceptional case, only 'null' value is allowed for all reference type.
    System.out.print("\nnums3 = " + nums3);
```

Similarly, the only values that you can get from a type declared with a super wild card are those of type Object, which is a super type of every reference type:

```
List<Object> objs2 = List.of(1, "two");
    List<? super Integer> ints4 = objs2;
    String str = "";
    for(Object ob : ints4){
       str += ob.toString();
    }
    System.out.print("\nstr = " + str);
```

You may find it helpful of think of **? extends T** as being an unknown type in an interval bounded by type of null below and by T above(Where the type of null is a subtype of every reference type.) Similarly, you may think of **? super** T as being an unknown type in interval bounded by T below and by object above.



It is tempting to think that an extends wild card ensures immutability, or at least un-modifiability but it does not.

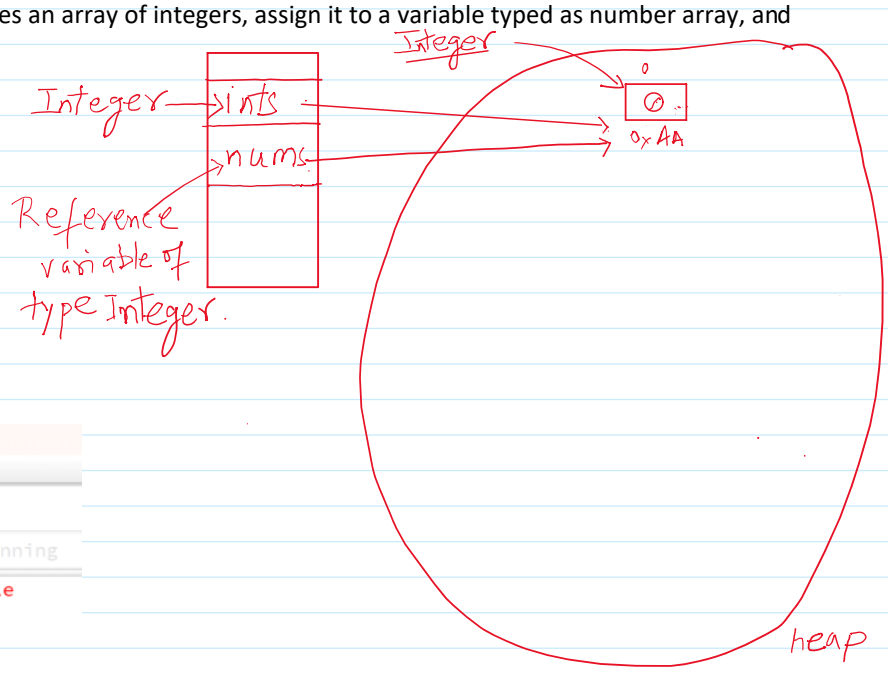
String is final and can have no subtypes. You might expect that list. <String > is same type as List <? extends String > But in fact that former is a subtype of the latter, not the same type as can be seen by an application of our principles. The substitution principle tells us it is a subtype because it is fine to pass a value of the former type where the latter is expected. The get and put principal tells us that it is not the same type because we can add a string to a value of the former type, but not the latter.

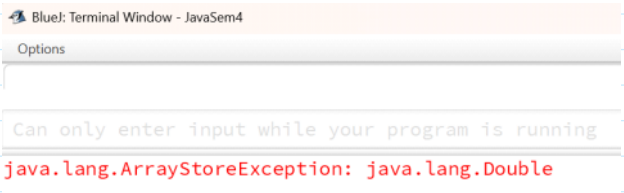
Arrays

It is instructive to compare the treatment of lists and arrays in Java. Keeping in mind the substitution principle and the get and put principle.
In Java, array subtyping is covariant, meaning that type S [] is considered to be the subtype of T [] whenever S is a subtype of T. Consider the following code fragment. Which allocates an array of integers, assign it to a variable typed as number array, and then attempts to store a double in the array:

```
Integer[] ints = {0};
Number[] nums = ints;
nums[0] = 3.14;
```

array store exception.





If it were to succeed, the variable ints as integer would be pointing at an array containing a Double. And once again, Java's type system would be broken.
Where is the problem? Since Integer is considered a subtype of Number. According to the substitution principle, the assignment on the second line must be legal. Instead, the problem caught at runtime on the 3rd line. When an array is allocated (as on the first line) it is tagged with its reified type (a runtime representation of its component type, in this case integer), and every time a value is stored in the array (as on the 3rd line) and array store exception is raised. If the reified type is not compatible with the assigned value (in this case, a double cannot be stored into an array of Integer).

In contrast, the subtyping relation for generics is invariant, meaning that the type List<S> is not considered to be subtype of List<T> except in the trivial case where S and T are identical. Here is a code fragment analogous to the preceding one with list replacing arrays.

```
List<Integer> ints = Arrays.asList(0);
List<Number> nums = ints;
nums.set(0, 3.14);
```

Error: compile-time.

Since List<Integer> is not considered to be a subtype of List<Number > the problem is detected on the 2nd line, not the 3rd, and it is detected at compile time, not runtime.
Wildcards reintroduce covariant subtyping for generics in that type List<S > is considered to be the subtype of List<? extends T > when S is a subtype of T. Here is the 3rd variant of the fragment:

```
List<Integer> ints = Arrays.asList(0);
List<? extends Number> nums = ints;
nums.set(0, 3.14); //Error: incompatible types: double cannot be converted to capture#1 of ? extends java.lang.Number
```

As with arrays, the 3rd line is in error, but in contrast to arrays, the problem is detected at the compile time, not runtime. The assignment violates the get and put principal. Because you cannot put a value into a type declared with an **extends** wild card character.

Wild card also introduced covariance subtyping for generics in that type, **List<S>** is considered to be a subtype of List<? super T > when S is a super type of T (As opposed to a subtype). Arrays do not support covariant subtyping. For instance recall that the method Count accepted a parameter of type Collection<? super Integer > and filled it with integers. There is no equivalent way to do this with an array. *Since Java does not permit you to write (? super integer) [].*

```
public static void count(Collection<? super Integer> ints, int n){
    for(int i = 0; i < n; i++){
        ints.add(i);
    }
}
```

Detecting problems at compile time rather than A runtime brings two advantages, one minor and one major. The minor advantage is that it is more efficient. The system does not need to carry around a description of the element type at runtime, and the system does not need to check against this description every time an assignment into an array is performed. The major advantage is that it is common family of errors is detected by the compiler. This improves every aspect of the program's life cycle.: coding, debugging, testing and maintenance are all made easier, quicker and less expensive.

Apart from the fact that typing errors are caught earlier. There are many other reasons to prefer collection classes to arrays. Collections are more flexible than arrays. The only operation supported on arrays are to get or set a component, and a number of elements they can contain is fixed. Collection supports many additional operations, including testing for containment, adding and removing elements, and combining two collections. Collections may be list (where order is significant and elements may be repeated and further operations are available) Sets (where order is not significant and element may not be repeated) Or queues (Predominantly used in workflow situations) And a number of representations are available, including arrays, linked list trees, and hash tables. Specialized collections are available for situation requiring efficient concurrent access. Finally, although this is not an inherent advantage of Collection over Arrays. The convenience class Collections offer operations to rotate or shuffle a list, to find the maximum of a collection, to make a collection unmodifiable or synchronized, and others. Many more than are provided by the corresponding convenience class Arrays.

Nonetheless, there are situation in which arrays are preferable to collections. In particular, arrays of primitive type are much more efficient than either arrays or collection of reference types, since they don't involve boxing and they use less memory with much better spatial locality. Further assignments into a primitive array need not check for an array store exception. Because arrays of primitive type do not have subtypes. These advantages may lead you to replace collection with arrays in performance critical code sections. As always, you should measure comparative performance to justify such a design, bearing in mind that future compiler and library design improvements may change the relative performance balance. Finally, in some cases arrays may be preferable for reason of compatibility.

To summarize, it is better to detect errors at compile time rather than runtime, but Java arrays are forced to detect certain errors at runtime by the decision to make array subtyping covariant. **Was this a good decision?** Before the advent of Generics, it was absolutely necessary. For instance, look at the following methods which are used to sort any array or to fill an array with a given value.

```
public static void sort(Object[] a);
public static void fill(Object[] a, Object val);
```

Thanks to convenience, these methods can be used to sort or fill arrays of any reference type. Without covariance and without Generics, there would have been no way to declare methods that apply for all types. With Generics, however, covariant arrays become unnecessary: These methods could now have the following signatures directly stating that they work for all types:

```
public static <T> void sort(T[] a);
public static <T> void fill(T[] a, T val);
```

In some sense, covariant arrays are an artifact of the lack of generics in earlier version of Java. Once you have generics,

covariant arrays are probably the wrong decision choice, and the only reason for retaining them is backward compatibility.

Wildcards Versus Type Parameters

The contains method checks whether a collection contains a given object and its generalization, containsAll, checks whether a collection contains every element of another collection. This sections presents two alternative approaches to giving generic signatures for these methods. The first approach uses wildcards and is the one used in the Java Collections Framework. The second approach uses type parameters.

Wildcards: Here are the types that the methods have in Java with generics:

```
interface Collection<E>{
    …
     public boolean contains(Object o);
     public boolean containsAll(Collection<?> c);
    …
}
```

The first method does not use generics at all. The second method is our first site of important abbreviation. The type collection <?> stands for:

Collection<? extends Object>

Extending Object is one of the most common uses of wildcards, so it makes sense to provide a short form of writing it.

These methods let us test for membership and containment:

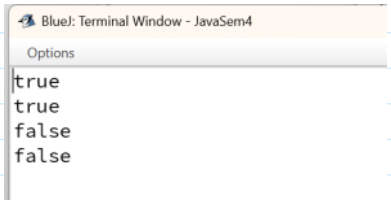
```
package GenericsAndCollections;
import java.util.*;

/**
 * Write a description of class Test here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Test
{
   public static void main(String args[]){

      Object obj = "one";
      List<Object> objs = List.of("one", 2, 3.5, 4);
      List<Integer> ints = List.of(2, 4);

      System.out.print("\f"+ objs.contains(obj)); //true
      System.out.print("\n" + objs.containsAll(ints)); //true
      System.out.print("\n" + ints.contains(obj)); //false
      System.out.print("\n" + ints.containsAll(objs)); //false
   }
}
```

Output:




```
package GenericsAndCollections;
```

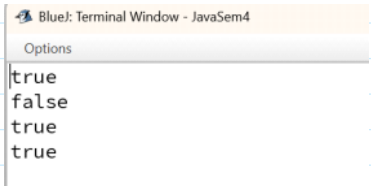
```
import java.util.*;

/**
 * Write a description of class Test here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Test
{
   public static void main(String args[]){

      //Object obj = "one";
      Object obj = 1;
      List<Object> objs = List.of(1, 3);
      List<Integer> ints = List.of(1, 2, 3, 4);

      System.out.print("\f"+ objs.contains(obj)); //true
      System.out.print("\n" + objs.containsAll(ints)); //false
      System.out.print("\n" + ints.contains(obj)); //true
      System.out.print("\n" + ints.containsAll(objs)); //true
   }
}
```

Output:



Type Parameters: You might reasonably choose an alternative design for collections. A design in which you can only test containment of subtypes of the elements type:

```
interface MyCollection<E> { //alternative design
    …
     public boolean contains(E o);
     public boolean containsAll(Collection<? extends E> c);
    …
}
```

Say we have a class MyList that implements MyCollection. Now the tests are legal only one way around:

```
Object obj = "one";
MyList<Object> objs = MyList.of("one", 2, 3.5, 4);
MyList<Integer> ints = MyList.of(2, 4);
System.out.print("\n" + objs.contains(obj));
System.out.print("\n" + objs.containsAll(ints));
System.out.print("\n" + ints.contains(obj)); //Compile-time error
System.out.print("\n" + ints.containsAll(objs)); //Compile-time error
```

The last two tests are illegal because the type declaration now require that we can only test whether a list contains an element of a subtype of that list. So we can check whether a list of objects contains a list of integers, but not the other way around.

The library designers choose wild cards over type parameters. In case of contains and contains all and other methods in Collection, List and Map that tests for, or remove values from, the collection. There are arguments both for and against this

choice.

Wild Card Capture

When a generic method is invoked, the type parameter may be chosen to match the unknown type represented by a wild card. This is called wild card capture.

Consider the method reverse in the convenience class java.util.Collections, which accepts a list of any type and reverses it. It can be given either of the following two signatures, which are equivalent:

```
public static void reverse(List<?> list);
public static <T> void reverse(List<T> list);
```

The wild card signature is slightly shorter and clearer, and is one used in the library.

If you use the second signature, it is easy to implement the method.

Type is known, because we capture the type as <T>.

```
public static<T> void reverse(List<T> list){
    List<T> tmp = new ArrayList<>(list);
    for(int x = 0; x < list.size(); x++){
        list.set(x, tmp.get(list.size() - 1 - x);
    }
}
```

That this copies the argument into a temporary list and then writes from the copy back into the original in reverse order.

If you try to use the first signature with similar method body, it won't work.

Type is not known.

```
public static void reverse(List<?> list){
    List<Object> tmp = new ArrayList<>(list);
    for(int x = 0; x < list.size(); x++){
        list.set(x, tmp.get(list.size() - 1 - x);   //Compile time error
    }
}
```

Now it is not legal to write from the copy back into original because we are trying to write from a list of objects into a list of unknown type. Replacing list <object > with list <? > won't fix the problem because now we have 2 list with (possibly different) unknown element types.
Instead, you can implement the method with the first signature by implementing a private method with the 2nd signature and calling the 2nd from the 1st.

```
public static void reverse(List<?> list){
    rev(list);
}
private static<T> void reverse(List<T> list){
    List<T> tmp = new ArrayList<>(list);
    for(int x = 0; x < list.size(); x++){
        list.set(x, tmp.get(list.size() - 1 - x);
    }
}
```

Here we say that the type variable T has captured the wild card. This is a generally useful technique when dealing with wildcards, and it is worth knowing.
Another reason to know about wild card capture is that it can show up in error messages even if you don't use the above technique. In general, each occurrence of a wild card is taken to stand for some unknown type. If the compiler prints an error message containing this type, it is referred to as capture of ?.

Restrictions on Wildcards

Wild card may not appear at top level in class instance creation expression (new) In explicit type parameters, in generic method calls or in super types (extends and implements).

Instance Creation: In a class instance creation expression, if the type is a parameterized type, then none of the type parameters may be wild cards. For example, the following are illegal:

```
List<?> list = new ArrayList<?>(); //Compile-time error

Map<String, ? extends Number> map = new HashMap<String, ? extends Number>(); //Compile-time error
```

This is usually not a hardship. The get and put principal tells us that if a structure contains a wild card. We should only get values out of it (if it is an extend wild card) or only put values into it (if it is a super wild card). For a structure to be useful, we must do both. Therefore, we usually create a structure at a precise type, even if we use wild card types to put values into or get values from the structure, as in the following example:

```
List<Number> nums = new ArrayList<Number>();
List<? super Number> sink = nums;
List<? Extends Number> source = nums;
for(int k = 0; k < 4; k++){
    sink.add(k);
}
int sum = nums.stream().mapToInt(Number::intValue).sum();
assert sum == 6;
```

Here wild card appears in the second and third lines but not in the first line that creates the list.

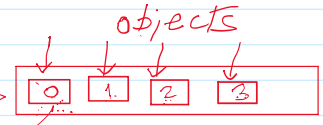
Stream<Number> **stream**()

Returns a sequential `Stream` with this collection as its source.

This method should be overridden when the {@link #spliterator()} method cannot return a spliterator that is `IMMUTABLE` , `CONCURRENT` , or *late-binding*. (See {@link #spliterator()} for details.)

implSpec - The default implementation creates a sequential `Stream` from the collection's `Spliterator` .
return - a sequential `Stream` over the elements in this collection
since - 1.8

nums.stream()



class scope resolution operator method reference

(Number::intValue)

IntStream **mapToInt**(ToIntFunction<? super Number> mapper)

Returns an `IntStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters

mapper - a non-interfering, stateless function to apply to each element

return - the new stream

Number::intValue is a method reference that acts as a ToIntFunction. For each Number object in the stream, it calls the intValue() method to extract its primitive int value. This effectively "unboxes" the Number objects into primitive ints.

From <https://www.google.com/search?client=firefox-b-d&q=int+sum+%3D+nums.stream%28%29.mapToInt%28Number%3A%3AintValue%29.sum%28%29%3B>

return - the new stream

IntStream Class.

int	sum()	Returns the sum of elements in this stream.

Only top level parameters in instance creations are prohibited from containing wild card. Nested wild cards are permitted. Hence the following is legal:

```
List<List<?>> lists = new ArrayList<List<?>>();
lists.add(List.of(1,2,3));
lists.add(List.of("four", "five"));
System.out.print("\nLists: " +lists);
```

Even though the list of lists is created at wildcard type, each individual list within it has a specific type: The first is a list of Integers and the second is a list of Strings. The wild card type prohibits us from extracting elements from the inner lists as any type other than Object, but since that is the type used by toString, this code is well typed.

One way to remember the restriction is that the relationship between wildcards and ordinary types is similar to the relationship between interfaces and class. Wildcards and interfaces are more general. Ordinary types and classes are more specific, and instance creation requires the more specific information. Consider the following three statements:

```
List<?> list = new ArrayList<Object>(); //OK
List<?> list = new List<Object>(); //Compile-time error
List<?> list = new ArrayList<?>(); //Compile-time error
```

The first is legal; the second is illegal because an instance creation expression requires a class, not an interface, and the 3rd is illegal because an instance creation expression requires an ordinary type, not a wild card.

My-Opinion

[You might wonder why this restriction is necessary. The Java designers had in mind that every wild card type is shorthand for some ordinary type. So they believed that ultimately every object should be created with an ordinary type. It is not clear whether this restriction is necessary, but it is unlikely to be a problem. (We tried hard to contrive a situation in which it was a problem and we failed.)]

Generic Method Call: If a generic method call includes explicit type parameters. Those type parameters must not be wild cards. For example, we have the following generic method:

```
class Lists{
    public static <T> List<T> factory(){
        return new ArrayList<T>();
    }
}
```

You may choose for the type parameters to be inferred, or you may pass an explicit type parameter.

```
List<?> list = Lists.factory();
List<?> list = Lists.<Object>factory();
```

If an explicit type parameter is passed, it must not be a wildcard:

```
List<?> list = Lists.<?>factory(); //compile-time error
```

As before, nested wild cards are permitted:

```
List<?> list = Lists.<List<?>>factory(); //ok
```

The motivation for this restriction is similar to the previous one. Again, it is not clear whether it is necessary, but it is unlikely to be a problem.

Supertypes: When a class instance is created, it invokes the constructor for its supertype. Hence, any restriction that applies to instance creation must also apply to supertypes. In a class declaration if the supertype or any super interface has type parameters. These types must not be wild cards.

```
 class AnyList extends ArrayList<?>{  //Compile-time error
      …
}
```

And so is this:

```
 class AnotherList implements List<?>{ // compile-time error
  …
}
```

But as before, nested wildcards are permitted.

```
 class NestedList extends ArrayList<List<?>>{  //Ok
      …
}
```

The motivation for this restriction is similar to the previous one. Again, it is not clear whether it is necessary, but it is unlikely to be a problem.

[my-opinion: Definition of behavioral subtyping in terms of the ability to substitute values of one type for the values of another without any change in program behavior. Applying this definition in reverse gives the design principle often called Liskov Substitution Principle.]

[Liskov Susbtitution Principle EXPLAINED SIMPLY | SOLID Principles #3]